

SPM: Performance Metrics & Laws

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Scalability Speedup Efficiency of a parallel program
- Strong and Weak Scaling
- Superlinear Speedup
- Amdahl's and Gustafson's Laws and their implications
- Computation to communication ratio

Basic Performance Metrics

- **Key definitions for system performance**

- **Latency (L):** is the time from the moment an input (request) arrives until the output (response) is ready to be delivered.
 - In the queueing theory and networking contexts, Latency is also known as **Response Time**. It includes any waiting time plus the service time
- **Completion time (T_c):** is the total time an application takes to complete all tasks, from start to delivery of all results
 - $L = T_c$ if there is only one task to compute for the application
 - Often referred to as the **makespan**
- **Service time (T_s):** the time required by the system to process a task, excluding waiting time
 - At steady state and full utilization, the time between two consecutive task completions approximates the service time.
- **Throughput :** it is the number of tasks completed per unit of time
 - At steady state and full utilization, it is the inverse of the service time ($\frac{1}{T_s}$);
- **Inter-arrival time (T_a):** the average time between two consecutive task arrivals at the system

Roadmap

- For the moment, we are primarily interested in the **completion time** of an application
- We want to understand how an application **scales** when parallelized across p processors
- The term processors refers to **processing elements** (i.e., thread/processes running on cores or machines, depending on the context)
- Scenario: we have a sequential version of an application whose completion time is T_{seq} . We want to estimate the speedup when running with **a parallelism degree p** (i.e., p processing elements) either on a machine with p cores, or on p identical machines
- Key questions:
 - How does speedup behave as p increases?
 - Is the parallelization efficient? That is, does it fully utilize available resources)?
 - What are the overheads?

Speedup definition

- **Speedup**: The speedup of a parallel program using p processors ($S(p)$) is defined as the ratio of the completion time of the sequential program (T_{c-seq}) to the completion time obtained running with p processors ($T_c(p)$)

$$S(p) = \frac{T_{c-seq}}{T_c(p)}$$

- It measures how much faster a program runs with more processors
- The best speedup one can expect, varying the number of processors, is **linear speedup**
 - If there is **no parallelization overhead**, the maximum speedup with p processors is $S(p) = p$
 - There are exceptions, which are referred to as **super-linear speedup** (discussed later)
- In general, because of the overhead introduced by the parallelization, the speedup is sub-linear with the number of processors used

Efficiency definition

- **Efficiency**: The efficiency of a parallel program parallelized using p processors ($E(p)$) is defined as the ratio between the speedup ($S(p)$) and the number of processors used p

$$E(p) = \frac{S(p)}{p} = \frac{T_{c-seq}}{T_c(p) \times p}$$

- Efficiency measures how effectively a parallel program utilizes the p resources employed
 - Its value lies between 0 and 1 (or between 0% and 100%). The ideal efficiency is 1
- $T_c(p) \times p$ is also called **cost of parallelization** (or simply **cost**)
 - If $T_c(p) \times p > T_{c-seq}$ then the parallelization overhead (i.e., communication, synchronization, idleness) impairs parallel efficiency and performance

Scalability definition

- **Scalability (relative speedup)**: It describes how well a parallel program can utilize an increasing number of processors, indicating whether adding more processors continues to yield performance gains. It is measured as:

$$S(p) = \frac{T_c(1)}{T_c(p)}$$

- What's the difference between Speedup and Scalability?
 - The key difference lies in what we use as baseline (next slide)

Scalability vs. Speedup

- The *scalability* and the *speedup* are related concepts. However, there is a significant difference:
 - The scalability considers $T_c(1)$ as the time obtained executing the **parallel implementation of the algorithm on a single processor**

$$\text{Scalability}(p) = \frac{T_{c-par}(1)}{T_{c-par}(p)}$$

This is often also referred to as *relative speedup*

- The speedup, instead, consider $T_c(1)$ as **the sequential version of the algorithm** (precisely, it should consider **the best sequential version solving the same problem**)

$$\text{Speedup}(p) = \frac{T_{c-seq}}{T_{c-par}(p)}$$

This is often referred to as *absolute speedup*

- Note that, in many cases $T_{c-seq} < T_{c-par}(1)$. Why?

Strong and Weak Scalability

- Does problem size affect scalability and speedup?
- **Strong scaling** (the problem size is fixed) ideal $T_c(p) = \frac{T_c(1)}{p}$ for a fixed PS
 - Linear scaling (i.e., $S(p) = p$) is difficult to achieve due to **Amdahl's law** (discussed later)
 - Performance depends on the amount of serial work (i.e., *the portion of the problem that cannot be parallelized*)
- **Weak scaling** (the problem size grows with p to keep the work per processor fixed)
 - The computational work (W) associated to the problem size (PS) remains (almost) constant per processor, i.e., the size function $PS(p)$ is such that $\frac{W(PS(p))}{p} \cong c$. In the common linear case, $PS(p) = p \cdot PS(1)$.
 - Therefore, ideal weak scaling is obtained when $T_c(PS(1), 1) = T_c(PS(p), p)$
 - Weak scaling assumes that, as the size of the problem grows:
 - the amount of serial work remains constant (or increases slowly)
 - the amount of communication/synchronization among processors remains constant (or increases slowly)

Weak-Scaling Efficiency

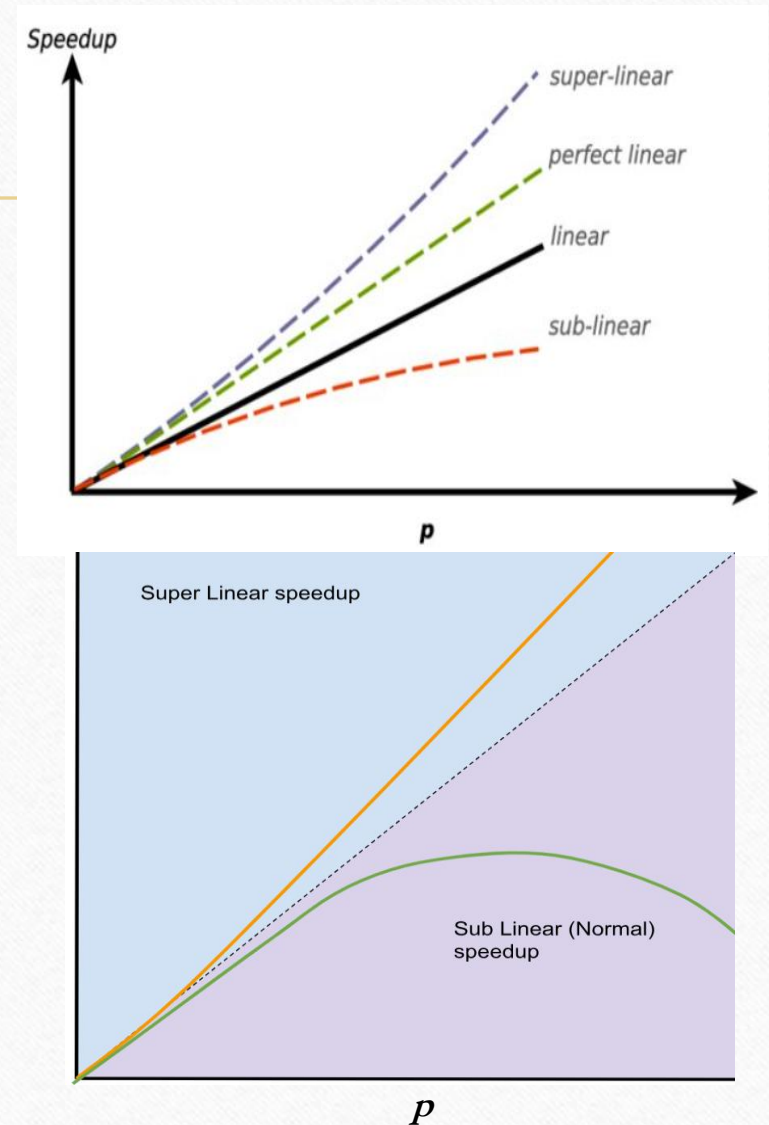
- **Weak-Scaling Efficiency**: measures how well execution time remains constant as both the number of processors and the problem size grow proportionally, keeping the workload per processor fixed

$$WSE(p) = \frac{T_c(PS(1), 1)}{T_c(PS(p), p)}$$

- Usually $WSE < 1$ even under ideal workload scaling (i.e., $\frac{W(PS(p))}{p} = c$), because execution time $T_c(PS(p), p)$ always exceeds $T_c(PS(1), 1)$ **due to communication and synchronization overhead**
- NOTE: in some cases, the problem size $PS(p)$ cannot be scaled linearly with p , i.e., $PS(p) \neq p \cdot PS(1)$ (discussed later)

Superlinear Speedup

- Can speedup be greater than p when using p processors?
 - Yes, it can happen, although it is rare to see it
- Two possible reasons:
 1. Unfair comparison with a naïve serial algorithm
 - E.g., the sequential version might not be compiled with optimization flags enabled
 - Can happen in some recursive algorithms (e.g., branch&bound algorithms)
 2. Cache/memory effects
 - More processors mean more memory and larger caches, leading to fewer cache misses and page swapping
- Unfortunately, quite often the speedup is sub-linear due to the parallel run-time overhead



Example and its analysis

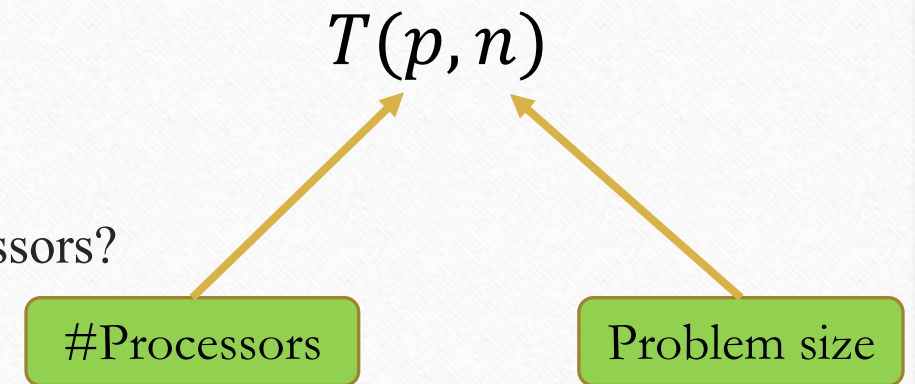
Input: Array A of n numbers **Output:** $\sum_{i=0}^{n-1} A[i]$
Task: Parallelize this problem by using p *processing elements (PEs)*

- **Assumptions (*):**
 - **Computation:** Each PE can add two numbers stored in its local memory in 1 sec
 - **Communication:** A PE can send data from its local memory to the local memory of any other PE in 3 sec (independently of the data size!)
 - **Input and Output:** At the beginning of the program, the whole input array A is stored in PE #0. At the end, the result must be gathered in PE #0
 - **Synchronization:** All PEs operate in a *lockstep* manner; i.e., they can either compute, communicate, or be idle
 - there is no *computation-to-communication overlap*

(*) These assumptions are **unrealistic**, but they are simple enough to allow us to perform a basic analysis!

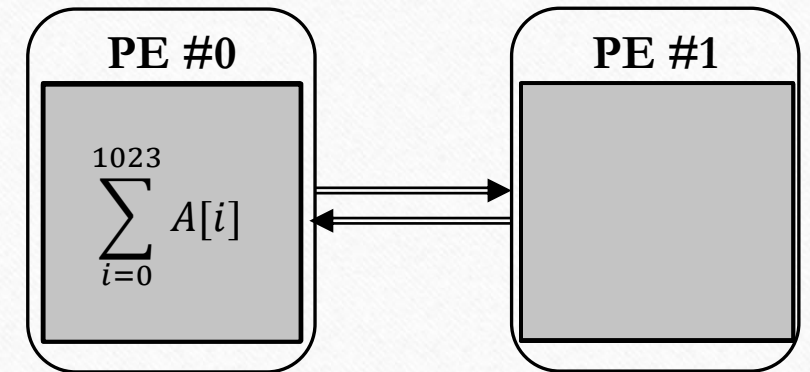
Example and its analysis (cont.)

- Establish sequential runtime as a baseline ($p = 1$):
 - For our example: $T(1, n) = n - 1$ sec
- What are we interested in?
 - **Speedup**: How much faster can we get with $p > 1$ processors?
 - **Efficiency**: Is our parallel program efficient?
 - **Scalability**: How does our parallel program behave when the number of processors varies keeping the problem size fixed (strong scaling) and when the problem size changes (weak scaling)?



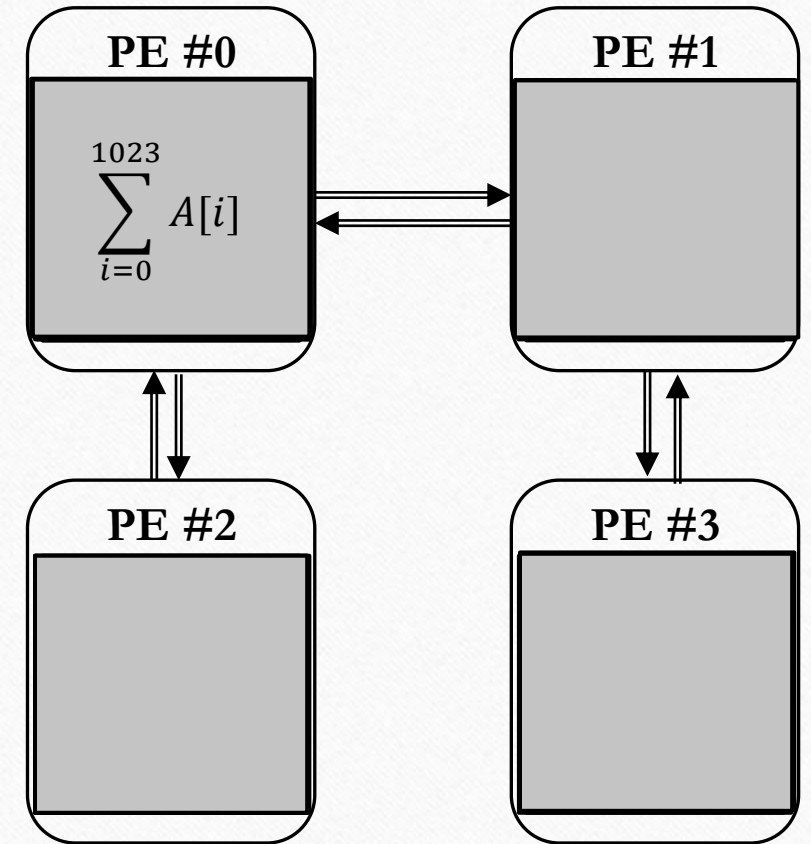
Example and its analysis (cont.)

- Establish runtime for 2 PEs ($p = 2$) and 1024 numbers ($n = 1024$):
 - $T(2,1024) = 3 + 511 + 3 + 1 = \mathbf{518 \text{ sec}}$
 - Speedup:** $T(1,1024)/T(2,1024) = 1023/518 = 1.975$
 - Efficiency:** $1.975/2 = 98.75\%$

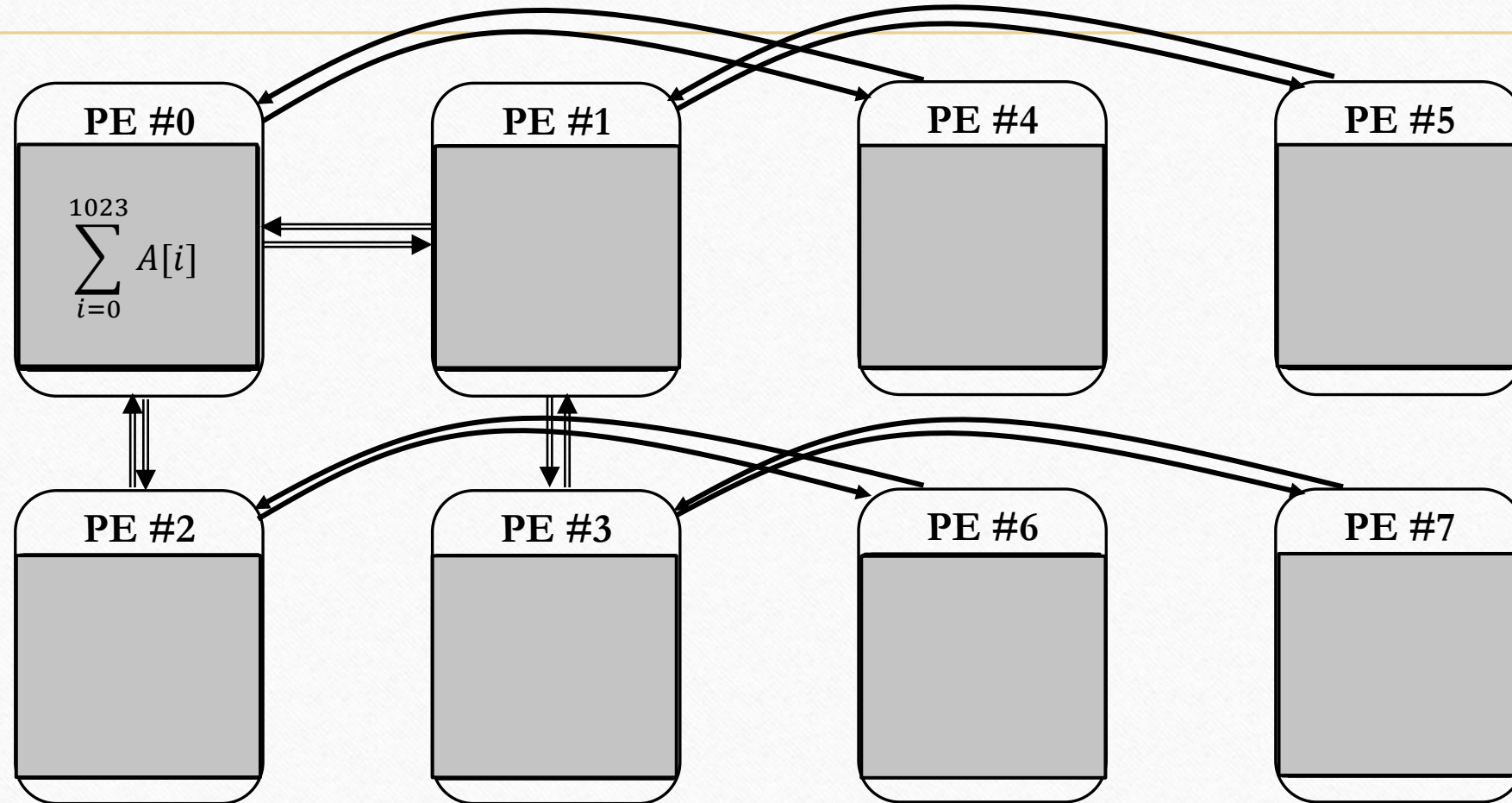


Example and its analysis (cont.)

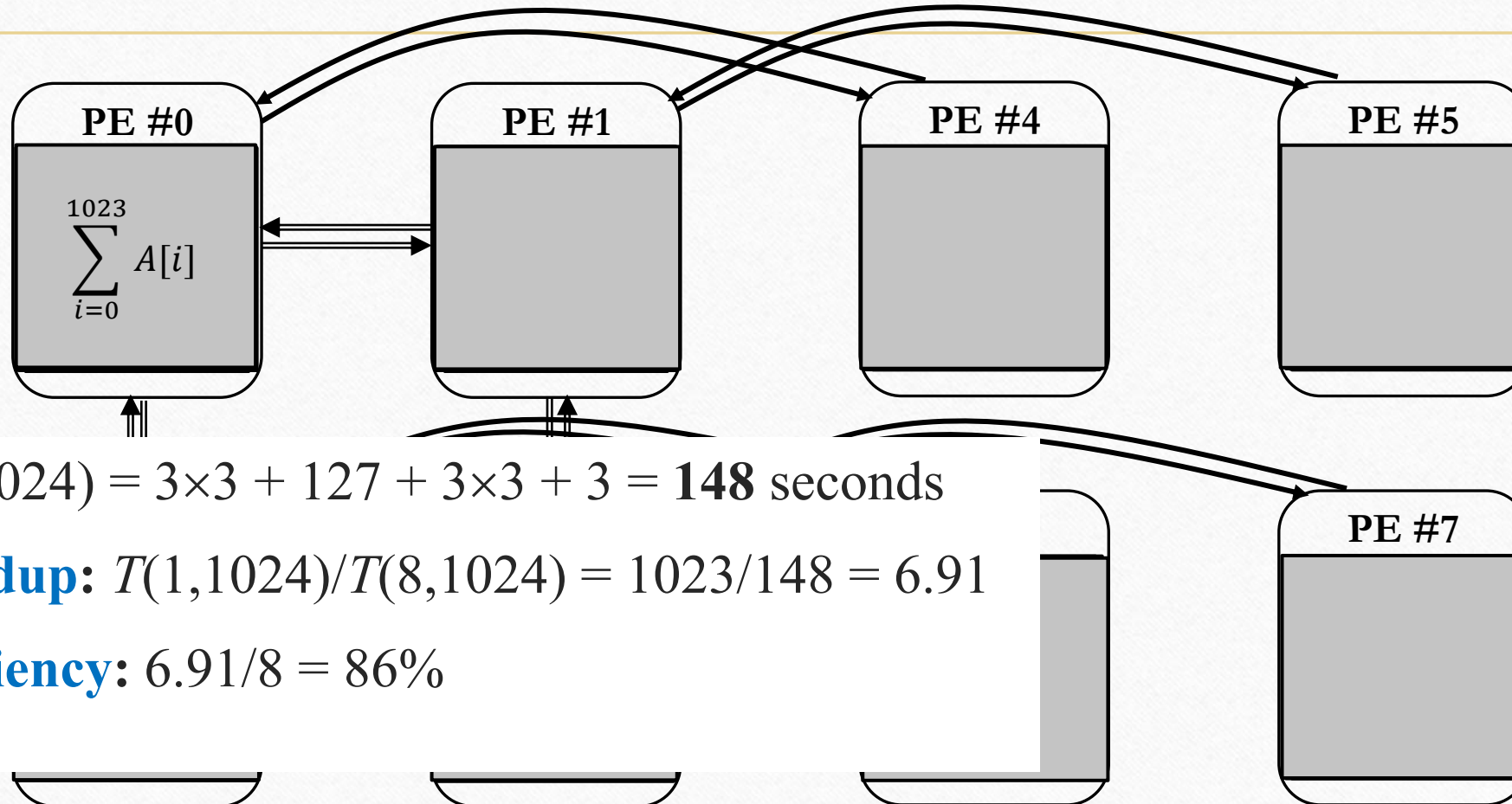
- $T(4,1024) = 3 \times 2 + 255 + 3 \times 2 + 2 = \mathbf{269}$ seconds
- **Speedup:** $T(1,1024)/T(4,1024) = 1023/269 = 3.803$
- **Efficiency:** $3.803/4 = 95.07\%$



Example and its analysis (cont.)



Example and its analysis (cont.)



- $T(8,1024) = 3 \times 3 + 127 + 3 \times 3 + 3 = \mathbf{148}$ seconds
- **Speedup:** $T(1,1024)/T(8,1024) = 1023/148 = 6.91$
- **Efficiency:** $6.91/8 = 86\%$

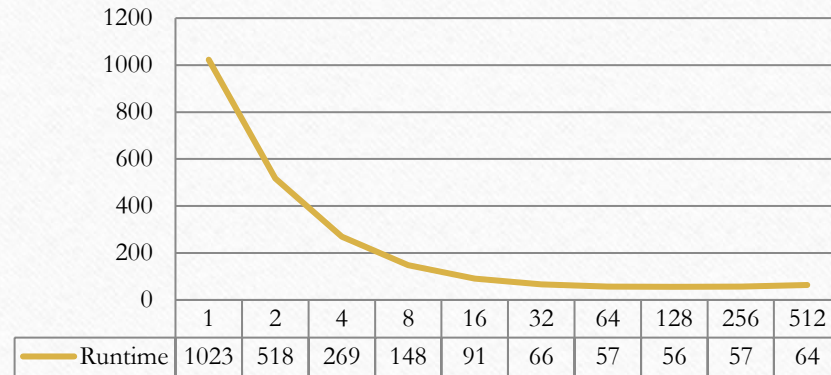
Example and its analysis (cont.)

- Timing analysis using $p = 2^q$ PEs and $n = 2^k$ input numbers:
 - Initial data distribution: $3 \cdot q$
 - Local computation : $n/p - 1 = 2^{k-q} - 1$
 - Collection partial results: $3 \cdot q$
 - Tree-based reduction to combine results: q

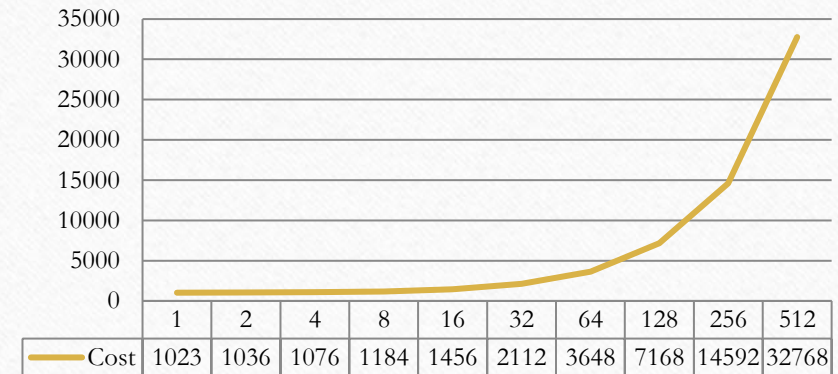
$$T(p,n) = T(2^q, 2^k) = 3q + 2^{k-q} - 1 + 3q + q = 2^{k-q} - 1 + 7q$$

Strong scalability Analysis ($n=1024$)

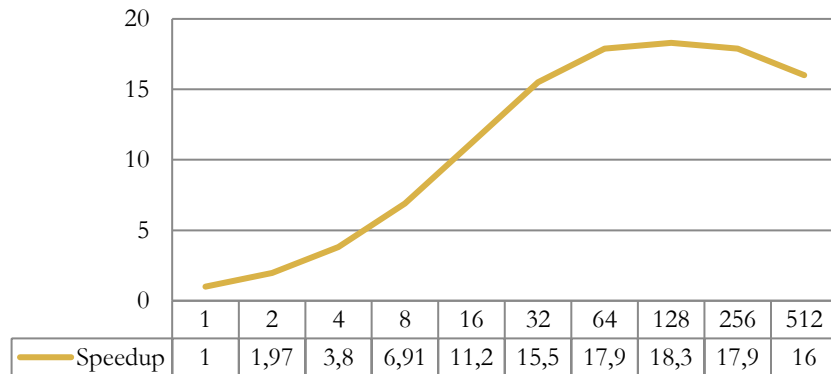
Runtime $T(1024,p)$



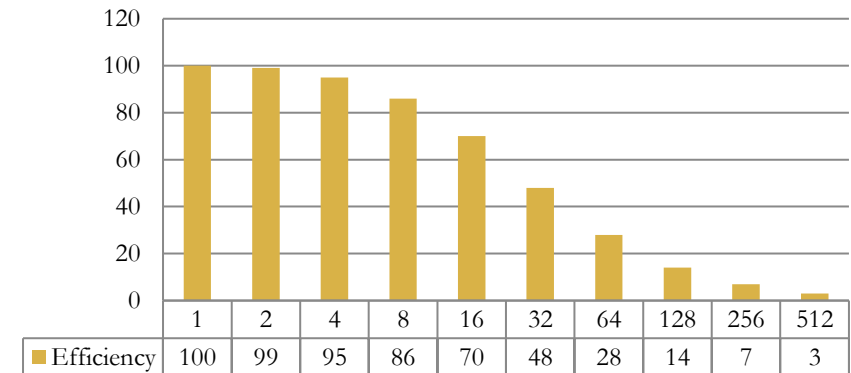
Cost = $T(1024,p) \times p$



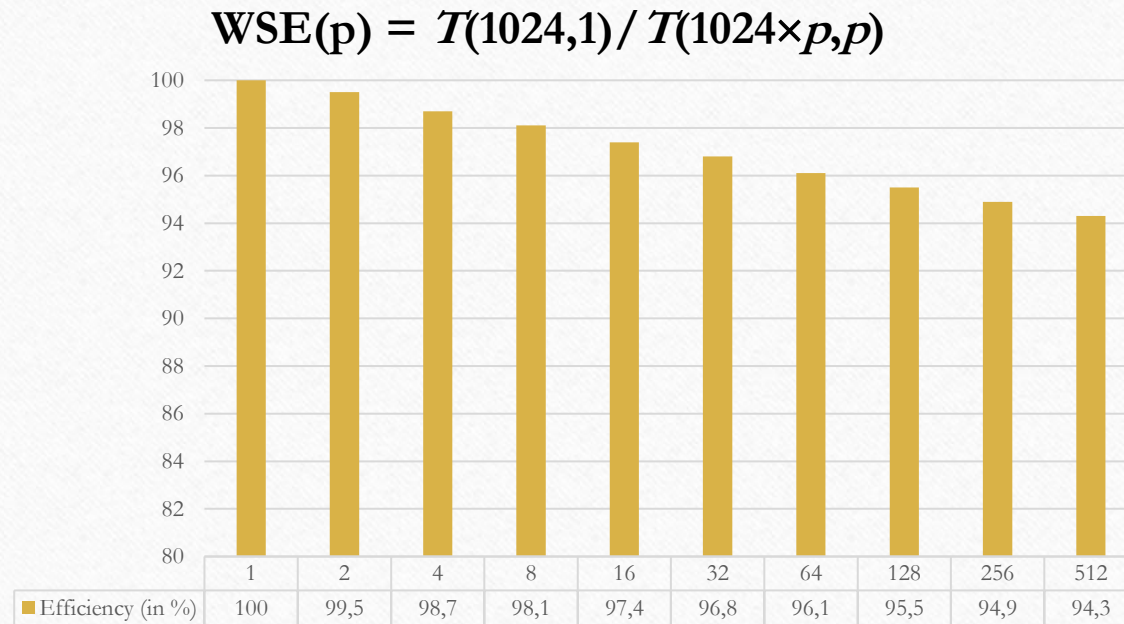
Speedup = $T(1024,1) / T(1024,p)$



Efficiency (in %) = $\text{Speedup} / p$



Weak scalability Analysis ($n=1024 \times p$)



Computation-to-Communication ratio

- Let $\alpha > 0$ and $\beta > 0$ be the amount of computation and communication/synchronization needed in our parallel application, respectively. In our simple example $\alpha = 1$ and $\beta = 3$
- $\gamma = \frac{\alpha}{\beta}$ is the **computation-to-communication ratio**
 - It measures the balance between computational workload and the parallelization overhead

$$T_{\alpha,\beta}(2^q, 2^k) = 2\beta q + \alpha(2^{k-q} - 1 + q)$$

$$S_{\alpha,\beta}(2^q, 2^k) = \frac{\alpha(2^k - 1)}{2\beta q + \alpha(2^{k-q} - 1 + q)}$$

$$S_{\gamma}(2^q, 2^k) = \frac{\gamma(2^k - 1)}{2q + \gamma(2^{k-q} - 1 + q)}$$

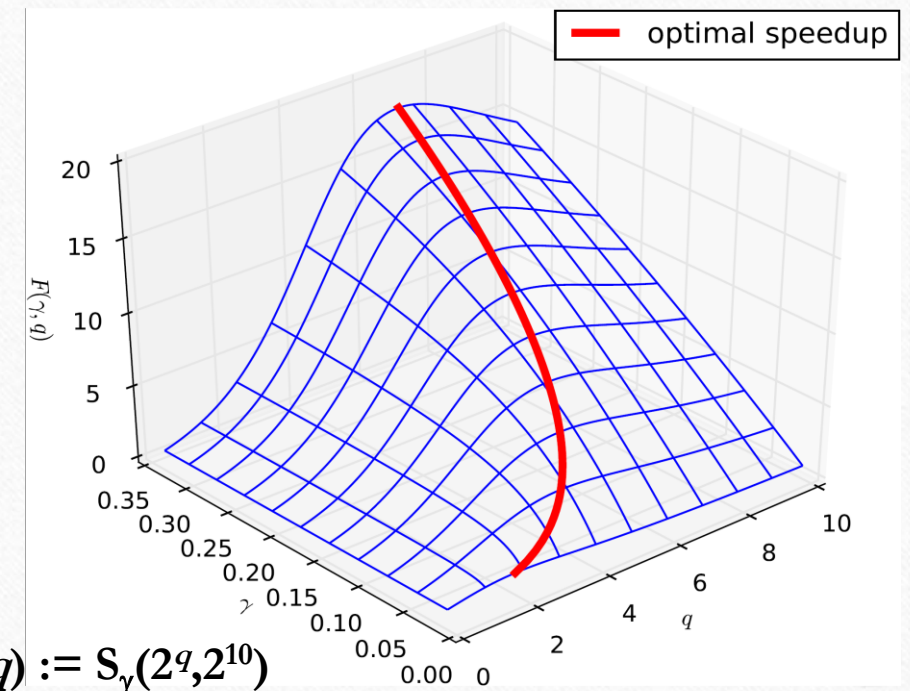


Diagram shows: $F(\gamma, q) := S_{\gamma}(2^q, 2^{10})$

Considerations

- Speedup and parallelization efficiency depend **on both the PEs used (p)** and the **computation-to-communication/synchronization ratio (γ)**
 - Speedup usually increases with the number of PEs used up to a local maximum, **then it tends to decrease as p increases because of the impact of the parallelization overhead**
 - There is thus an optimal number of PEs (p_{opt}). For $p > p_{opt}$ performance either worsens or does not increase
 - **Speedup is low if γ is low**. Usually, the longer the communication takes, the fewer PEs should be used.
 - The number (or frequency) of communications is also a critical factor influencing performance
 - **Efficiency is a monotonic function** in the number of PEs and the computation-to-communication ratio
 - Efficiency typically decreases as p increases (due to overheads)
 - Efficiency typically increase as γ increases (due to the minor impact of the comm./synch. Overheads)

Weak Scaling with non-linear problem size

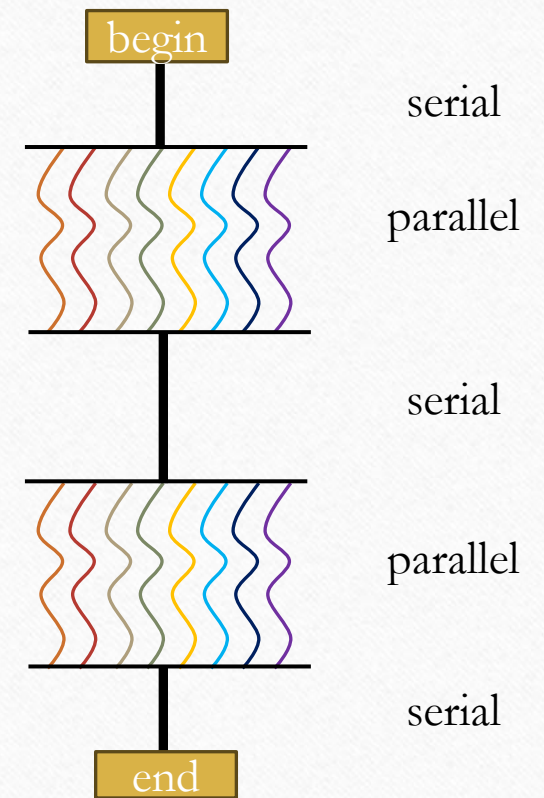
- We want to calculate the problem size S for computing weak scalability of a problem $P(p, S)$ that we can parallelize using p processors and which has a computational cost $W(S)$
- What is the value of S for $p = 1, 2, \dots, n$ if $W(S)$ does not grow proportionally with S ?
 - For example, for the matrix multiplication the input data size is $S = 2 \cdot N^2$ and the computational cost $W(S) = O(N^3)$ therefore $T_c(1, N^2) \neq T_c(p, (p \cdot N^2))$
- Let's suppose that the input size is N^2 and $W(N^2) = N^3$. For ideal weak scaling we want that $\frac{W(S)}{p} = c$ (for some constant c), thus in our example $N = c' \cdot p^{\frac{1}{3}}$ for some c'
- for $p = 1, 2, 4, 8, 16, 32$ and initial $N = 2^{10}$, the scaled values of N as p grows are in the table
- If we have a fixed input size (e.g., 1GB, 10GB, 100GB) we can determine p using a similar reasoning

p	Problem size (N)	
1	2^{10}	1024
2	$2^{10} \cdot 2^{\frac{1}{3}}$	1290
4	$2^{10} \cdot 2^{\frac{2}{3}}$	1625
8	2^{11}	2048
16	$2^{10} \cdot 2^{\frac{4}{3}}$	2580
32	$2^{10} \cdot 2^{\frac{5}{3}}$	3251

Amdahl's Law

- **Amdahl's law** describes the theoretical limit on the achievable speedup when using multiple processors for a fixed problem size
- It states that the speedup is fundamentally limited by **the fraction of sequential code** (the portion of the code that cannot be parallelized)
- Almost every program has a fraction of the code that cannot be parallelized. This fraction must be executed sequentially, even in a parallel implementation
- Thus, **Amdahl's law** establishes an **upper bound** on achievable speedup
 - This bound assumes perfect parallelization of the parallelizable portion (i.e., no communication overhead, perfect load-balancing)
 - In practice, **this upper bound is often unrealistic** (discussed later)

Typical parallel program



Derivation of Amdahl's Law

$$T_c(1) = T_{ser} + T_{par}$$

part of the program that does not benefit from parallelization

part of the program that can benefit from parallelization

- We now assume that the best possible speedup we can achieve is linear (no super-linear speedup)
- Then, we derive an upper bound for achievable speedup

$$T_c(p) \geq T_{ser} + \frac{T_{par}}{p}$$

$$S(p) = \frac{T_c(1)}{T_c(p)} \leq \frac{T_{ser} + T_{par}}{T_{ser} + \frac{T_{par}}{p}}$$

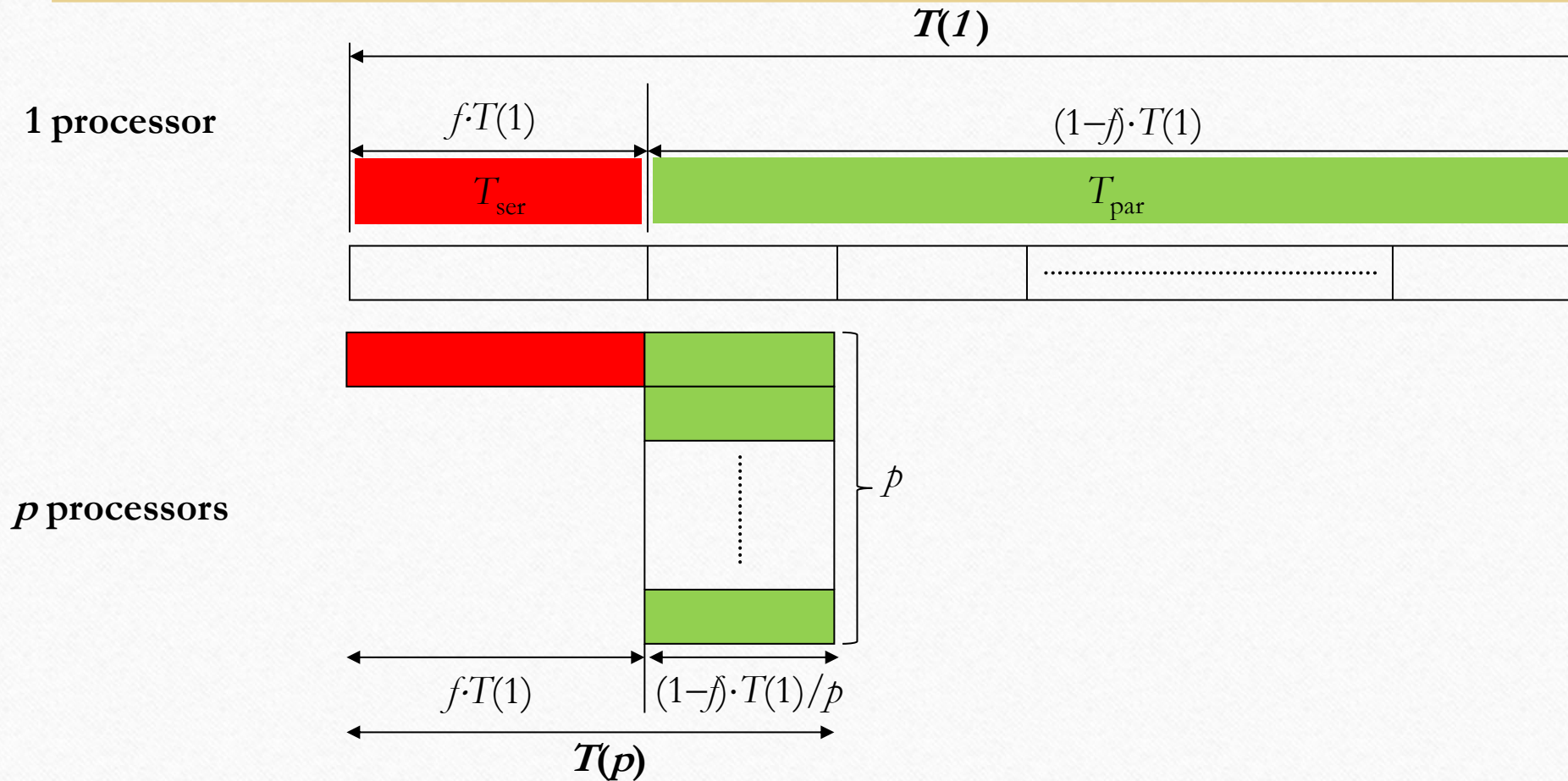
Derivation of Amdahl's Law

- Instead of using absolute execution times (T_{ser} and T_{par}), we now use their relative fraction
 - f is the serial fraction whereas $(1-f)$ is the parallelizable fraction
- Substituting f in the previously derived upper bound, the result is the Amdahl's Law
 - the upper bound for the speedup only depends on f and p

$$T_{ser} = f \cdot T_c(1) \quad T_{par} = (1 - f) \cdot T_c(1); \quad (0 \leq f \leq 1)$$

$$S(p) = \frac{T_c(1)}{T_c(p)} \leq \frac{T_{ser} + T_{par}}{T_{ser} + \frac{T_{par}}{p}} = \frac{f \cdot T_c(1) + (1 - f) \cdot T_c(1)}{f \cdot T_c(1) + \frac{(1 - f) \cdot T_c(1)}{p}} = \frac{1}{f + \frac{(1 - f)}{p}}$$

Amdahl's Law



Amdahl's Law

$$S(p) \leq \frac{1}{f + \frac{(1-f)}{p}}$$

- The sequential fraction of code, f , is a unit-less measure between 0 and 1
- Amdahl's Law can be used to predict the performance of parallel programs
- Examples:
 - 95% of a program's execution time occurs inside a loop that can be executed in parallel. What is the maximum speedup we should expect from a parallel version of the program executing on 6 CPUs?
 - 10% of a program's execution time is spent within inherently sequential code. What is the limit to the speedup achievable by a parallel version of the program?

$$S(6) \leq \frac{1}{0.05 + \frac{(1-0.05)}{6}} = 4.8$$

$$S(\infty) \leq \lim_{p \rightarrow \infty} \frac{1}{0.1 + \frac{(0.9)}{p}} = 10$$

Key message: Even with infinite processors, a 10% sequential fraction limits speedup to 10×

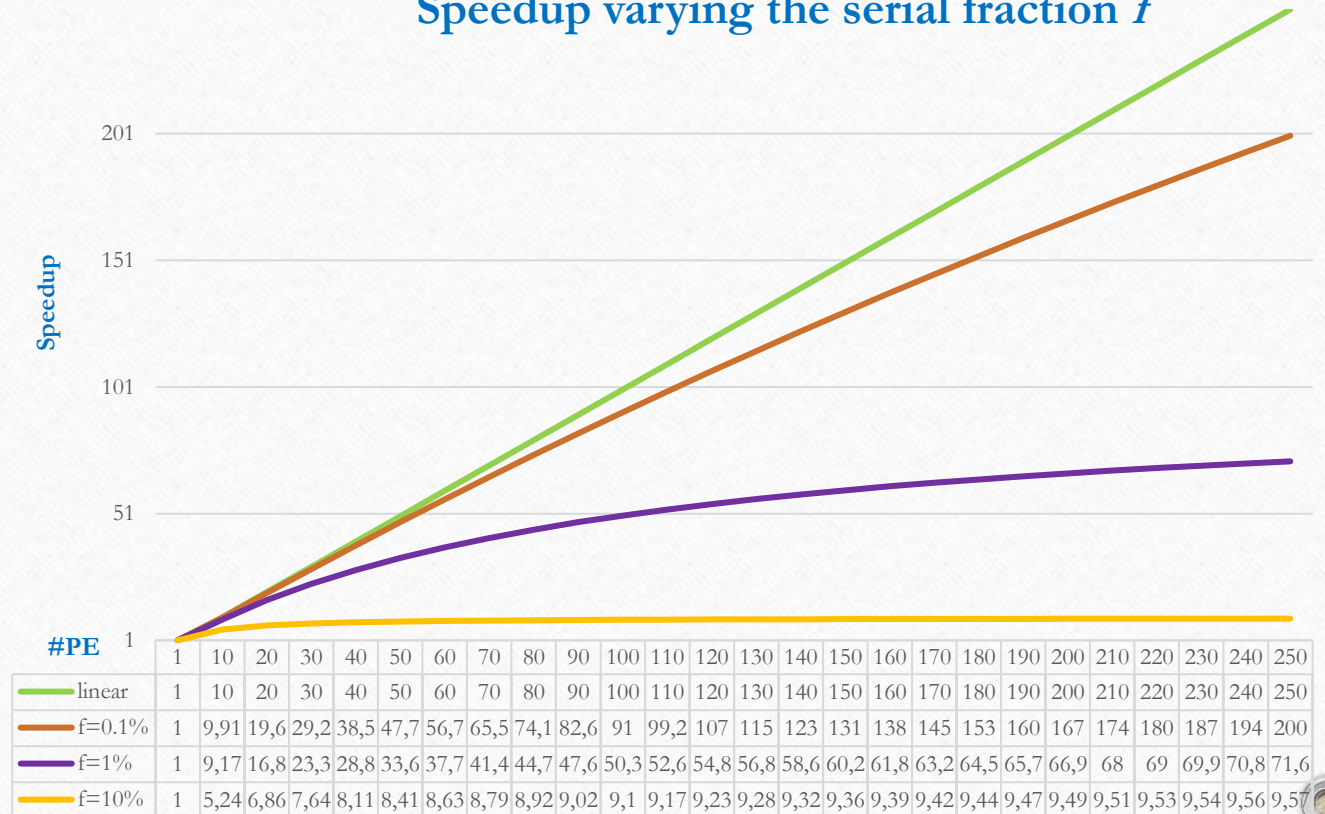
This shows why reducing the sequential portion is critical for (strong) scalability.

Amdahl's Law defeats strong scaling

$$S(p) \leq \frac{1}{f + \frac{(1-f)}{p}}$$

- For large p (i.e., $p \rightarrow \infty$) $S(p) \rightarrow \frac{1}{f}$
- Therefore, **strong scaling is fundamentally limited by the serial fraction**
 - If $f=1\%$ then $\max_p S(p, f) = 100$
 - If $f=10\%$ then $\max_p S(p, f) = 10$

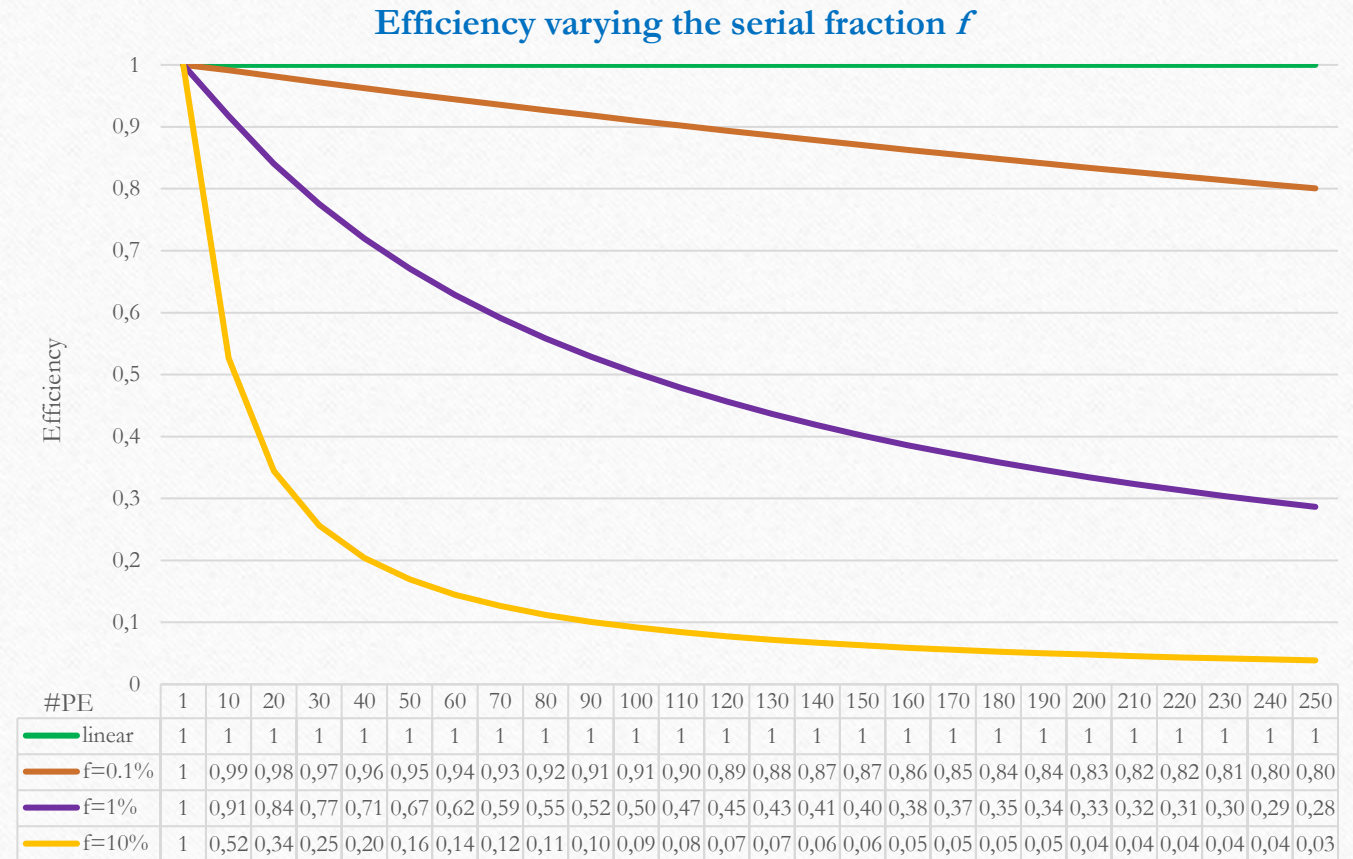
Speedup varying the serial fraction f



Efficiency under Amdahl's Law

- What is the shape of the efficiency curves when varying f ?

$$E(p) = \frac{S(p)}{p} = \frac{1}{p \cdot f + (1 - f)}$$



Amdahl's Law is far too optimistic

- **Why does Amdahl's Law establish an “unrealistic” upper bound?**
- Amdahl's Law does not account for:
 - **Communication cost**: even in the simplest parallel model, some communications or data movement are present
 - **Overheads**: every parallel implementation introduces overheads due to synchronization and coordination
- It is not true that we can arbitrarily increase p and always improve the speedup
- A more plausible Amdahl's Law would be something like:

$$S(p) \leq \frac{1}{[f + O(p)] + \frac{(1-f)}{p}}$$

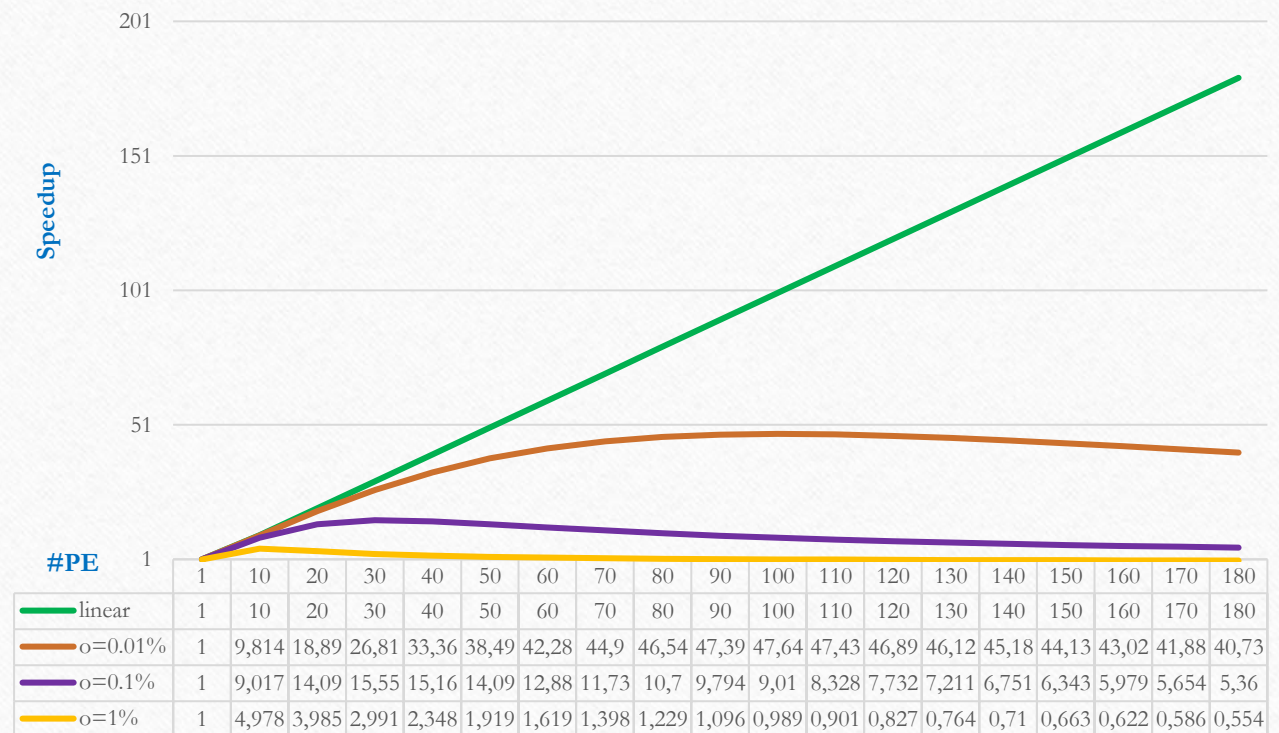
Where $O(p)$ is the additional serial fraction of time that includes **any source of overhead**, such as idle time, synchronization, and communication costs for running the application in parallel on p processors

- Example, logarithmic overhead $O(p) = c \cdot \log(p)$ for some constant c

Amdahl's Law with linear overheads

$$S(p) \leq \frac{1}{f + p \cdot o + \frac{(1-f)}{p}}$$

Speedup with $f=0.1\%$ and the overhead growing as $p \cdot o$



Scaled Speedup

- Amdahl's law only applies in situations where the problem size is fixed and the number of processors varies ($\Rightarrow$ **strong scalability**)
- However, when using more PEs, we often can solve larger problems ($\Rightarrow$ **weak scalability**)
 - In this case, the time spent in the parallelizable part (T_{par}) may grow faster in comparison to T_{ser}
 - This change the speedup analysis significantly
- **Scaled Speedup**, incorporates such scenarios when calculating the achievable speedup
- We derive **a more general** law that enables us to study the scaling with respect to the problem's complexity/size
 - **Gustafson's Law** is a particular case of the Scaled Speedup that can be used to predict the theoretically achievable speedup using multiple processors when the parallelizable part scales linearly with the problem size (Weak scaling), **while the serial part remains constant**

Derivation of Scaled Speedup Law

$$T_{\sigma\pi}(1) = \sigma \cdot T_{\text{ser}} + \pi \cdot T_{\text{par}} = \sigma \cdot f \cdot T_c(1) + \pi \cdot (1 - f) \cdot T_c(1)$$

σ : scaling function of the part of the program that does not benefit from parallelization with respect to the complexity of the problem size

π : scaling function of the part of the program that may benefit from parallelization with respect to the complexity of the problem size

$$S_{\sigma\pi}(p) = \frac{T_{\sigma\pi}(1)}{T_{\sigma\pi}(p)} \leq \frac{\sigma \cdot f \cdot T_c(1) + \pi \cdot (1 - f) \cdot T_c(1)}{\sigma \cdot f \cdot T_c(1) + \frac{\pi \cdot (1 - f) \cdot T_c(1)}{p}} = \frac{\sigma \cdot f + \pi \cdot (1 - f)}{\sigma \cdot f + \frac{\pi \cdot (1 - f)}{p}}$$

NOTE: Here σ and π represent how the serial and parallel portions scale with the problem

Derivation of Gustafson's Law

$$\gamma = \frac{\pi}{\sigma}$$

Ratio of the problem complexity scaling between the parallelizable part and the non-parallelizable part.

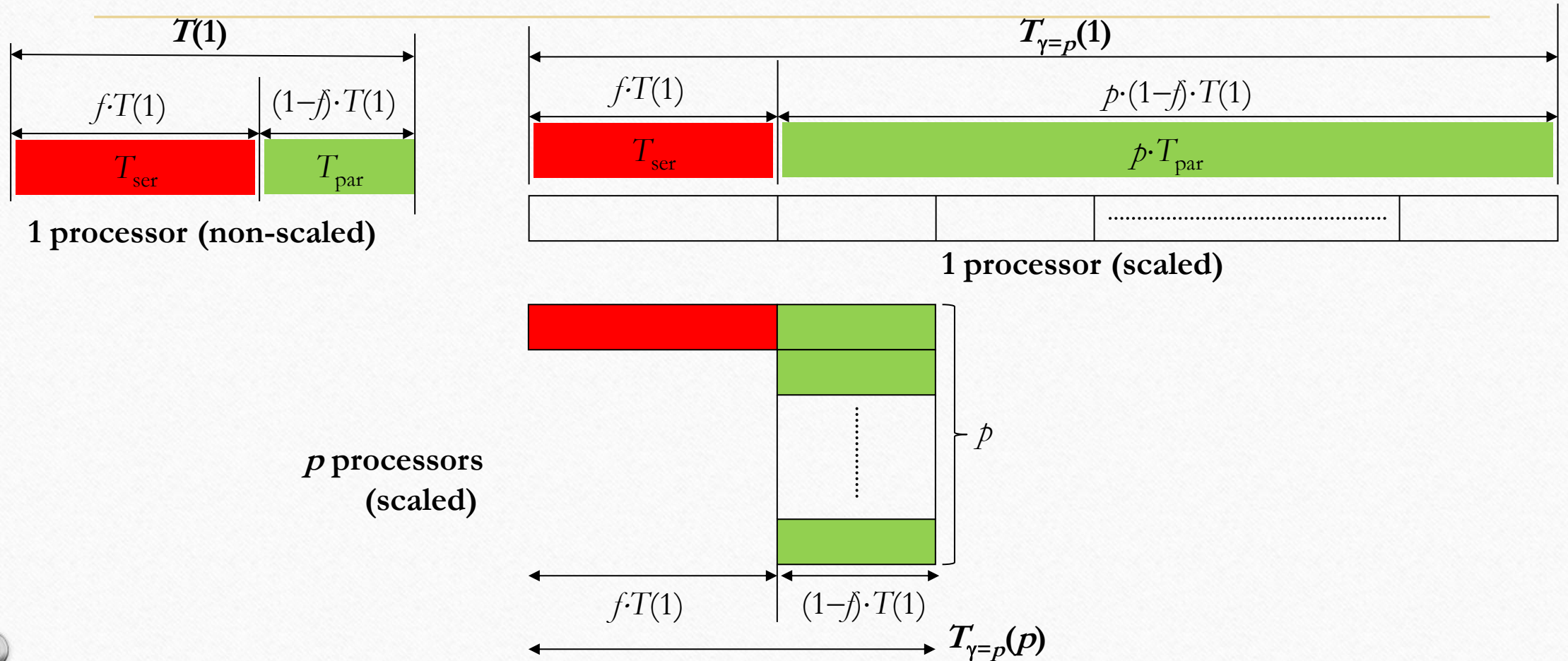
$$S_{\gamma}(p) \leq \frac{f + \gamma \cdot (1 - f)}{f + \frac{\gamma \cdot (1 - f)}{p}}$$

Scaled speedup as a function of γ

- Using different functions for γ yields to the following two notable cases:
 - $\gamma = 1$ (i.e. $\sigma = \pi$) $\Rightarrow$ **Amdahl's Law**
 - $\gamma = p$ (e.g. $\sigma = 1$; $\pi = p$), i.e., the parallelizable part grows linear in p while the non-parallelizable part remains constant $\Rightarrow$ **Gustafson's law**:

$$S(p) \leq f + p \cdot (1 - f) = p + f \cdot (1 - p)$$

Gustafson's Law



Scaled speedup example

- Suppose we have a parallel program that, for a given problem size PS, is 15% serial and 85% linearly parallelizable. Let us assume that when we scale PS, the absolute serial time remains constant (i.e., $\sigma = 1$) while the parallel portion grows linearly with the scaling factor (π)

- How much speedup can we achieve if we use 50 processors without scaling PS?

$$S_{\gamma=1}(50) \leq \frac{f + \gamma \cdot (1 - f)}{f + \frac{\gamma \cdot (1 - f)}{p}} = \frac{1}{0.15 + \frac{0.85}{50}} = 5.99$$

- Suppose now that PS is scaled by a factor of 100. Which is the speedup with 50 processors?

$$S_{\gamma=100}(50) \leq \frac{f + \gamma \cdot (1 - f)}{f + \frac{\gamma \cdot (1 - f)}{p}} = \frac{0.15 + 100 \cdot 0.85}{0.15 + \frac{100 \cdot 0.85}{50}} = 46.03$$

Questions

- Assume that you want to write a program that should achieve a speedup of 100 using 128 processors.
 - What is the maximum sequential fraction of the program when this speedup should be achieved under the assumption of **strong scalability**?

$$100 = \frac{1}{f + \frac{(1-f)}{128}} = \frac{128}{128 \cdot f + 1 - f} = \frac{128}{127 \cdot f + 1} \implies f = \frac{0.28}{127} = 0.0022 \quad (0.22\%)$$

- What is the maximum sequential fraction of the program when this speedup should be achieved under the assumption of **weak scalability** (whereby γ scales linearly)?

$$100 = 128 + f \cdot (1 - 128) = 128 - 127 \cdot f \implies f = \frac{28}{127} = 0.22 \quad (22\%)$$

Functional Dependency of Scaled Speedup

$S_\gamma(p)$ parametrized with $\gamma = p^\delta$

1/2

- The scaled speedup formula $S_\gamma(p) \leq \frac{f + \gamma \cdot (1-f)}{f + \frac{\gamma \cdot (1-f)}{p}}$ depends on how γ scales with p
- We can write $\gamma = p^\delta$ where:
 - $\delta = 0 \rightarrow \gamma = 1$ the problem size is fixed (Amdahl's law)
 - $\delta = 1 \rightarrow \gamma = p$ the problem size scales linearly with p (Gustafson's law)
 - $0 < \delta < 1$ intermediate scaling behavior
- The parameter δ characterizes how aggressively we scale the problem size as we add processors
- The choice of $\gamma = p^\delta$ is motivated by:
 - Power-law scaling in many real problems
 - Mathematical tractability and elegance (includes Amdahl and Gustafson)
 - Not so complex to estimate experimentally

Functional Dependency of Scaled Speedup

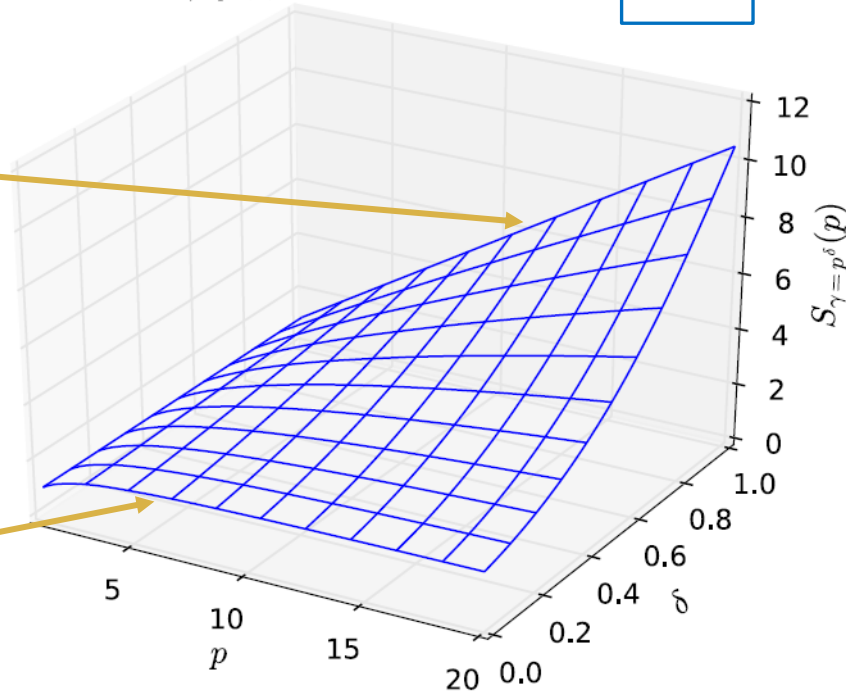
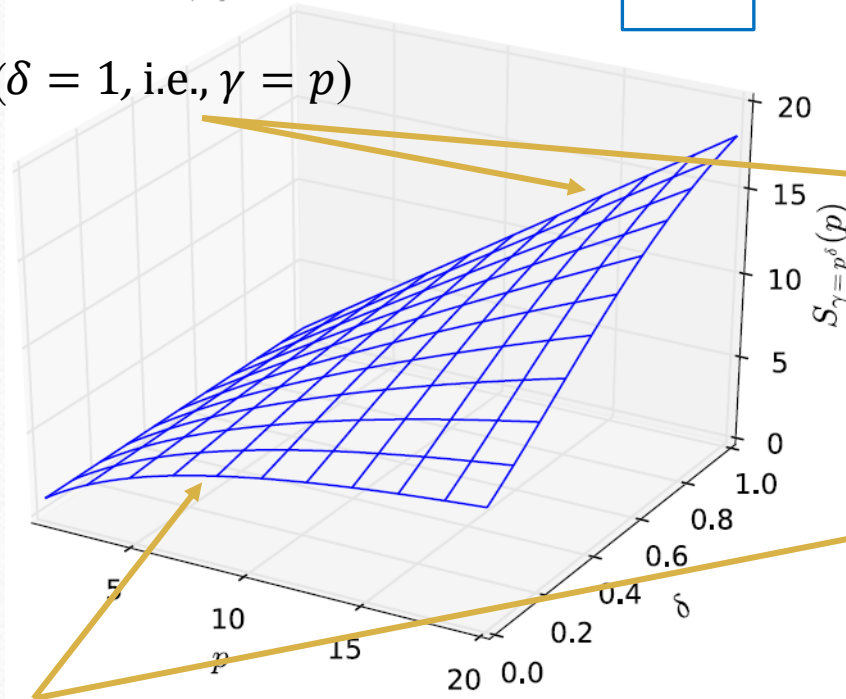
$S_\gamma(p)$ parametrized with $\gamma=p^\delta$

2/2

Speedup $S_{\gamma=p^\delta}(p)$ for sequential fraction $f=0.1$

Speedup $S_{\gamma=p^\delta}(p)$ for sequential fraction $f=0.5$

Gustafson ($\delta = 1$, i.e., $\gamma = p$)



Amdahl ($\delta = 0$, i.e., $\gamma = 1$)

Scaled Efficiency as a function of $\gamma=p^\delta$

- Let us consider again the scaled speedup and compute the efficiency

$$E_\gamma(p) = \frac{S_\gamma(p)}{p} \leq \frac{f+\gamma\cdot(1-f)}{p\cdot f+\gamma\cdot(1-f)} \text{ where } \gamma = p^\delta \quad \delta \in [0,1]$$

- $E_\gamma(p)$ is called **scaled efficiency** as a function of γ
- We can observe the limiting cases:
 - For $\delta = 0 \rightarrow \gamma = 1$ (Amdahl's law) $\lim_{p \rightarrow \infty} E_{\gamma=1}(p) = 0$, efficiency inevitably drops to zero as p grows, regardless the serial fraction f
 - For $\delta = 1 \rightarrow \gamma = p$ (Gustafson's law) $\lim_{p \rightarrow \infty} E_{\gamma=p}(p) = (1 - f)$, efficiency remains high if f is small

Iso-Efficiency Concept

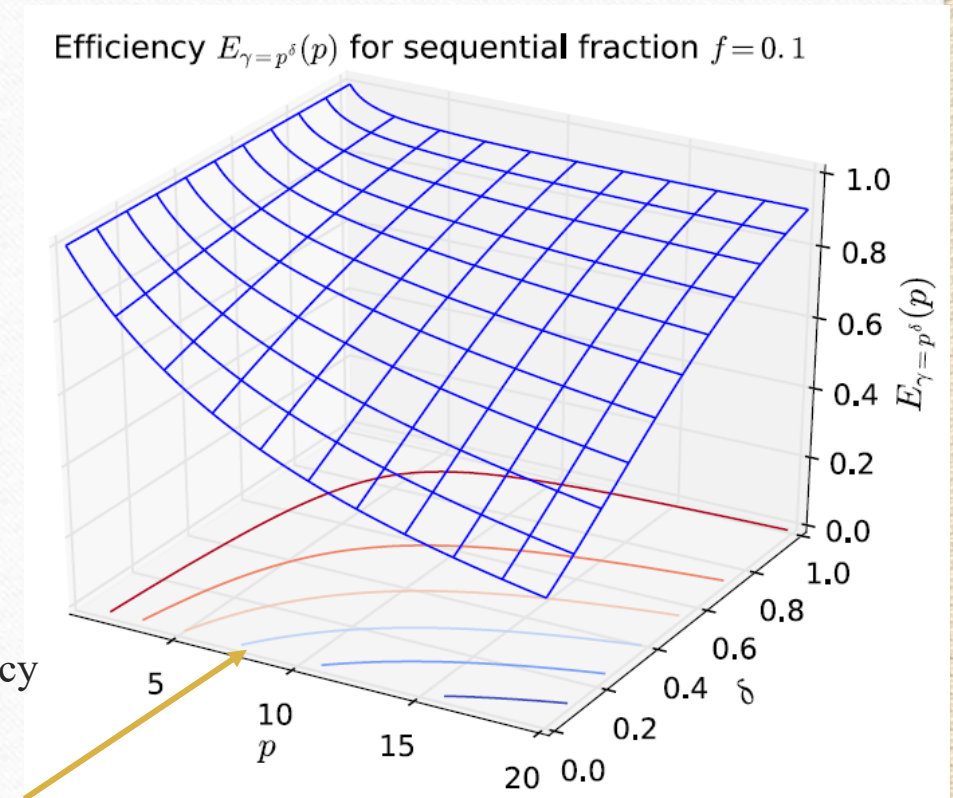
- Motivating questions:
 - How much must δ increase in order to preserve parallel efficiency for increasing p ?
 - How much does the parallelizable part of our problem need to grow to maintain the same efficiency with more processing?
- **Definition:** An **iso-efficiency** function describes how problem scaling (δ) must vary with the number of processors (p) to maintain a constant efficiency E_0
- Mathematically, for a target efficiency E_0 , we have
$$E_0 = \frac{f + p^\delta \cdot (1 - f)}{p \cdot f + p^\delta \cdot (1 - f)}$$
- In the (p, δ) space, there are curves along which the efficiency remains constant at E_0

Functional Dependency of Scaled Efficiency parametrized with $\gamma=p^\delta$

- For a target efficiency E_0 , the iso-efficiency relation is:

$$\delta(p, E_0) = \frac{\log[f \cdot (p \cdot E_0 - 1)] - \log[(1 - f) \cdot (1 - E_0)]}{\log p}$$

- Let's examine this with $f=0.1$ (i.e., 10% serial fraction), and $p=10$:
 - $E_0 = 0.9$ (90% efficiency) $\rightarrow \delta(10, 0.9) \approx 0.95$
 - To maintain 90% efficiency with 10 processor the problem must scale almost linearly (i.e., $\delta \approx 1$)
 - $E_0 = 0.5$ (50% efficiency) $\rightarrow \delta(10, 0.5) \approx 0.05$
 - Even with smaller scaling ($\delta \approx 0$) we can maintain 50% efficiency with 10 processors



Iso-efficiency curves, each curve represent one E_0 value

Suggested Readings

- Chapter 1 Section 1.1 of the «Parallel Programming Concept and Practice» book
- Chapter 2 Section 2.3 of the «Parallel Programming Concept and Practice» book
- John L. Gustafson «Reevaluating Amdahl's Law» Communications of the ACM
<https://dl.acm.org/doi/pdf/10.1145/42411.42415>
- A. Y. Grama, A. Gupta, V. Kumar «Isoefficiency: Measuring the Scalability of Parallel Algorithms and Architectures» IEEE <https://ieeexplore.ieee.org/document/242438>